TeamVault

Team Management System with Role-Based Access Control

Full-Stack MERN Application

Technology: MongoDB · Express.js · React · Node.js

Repository: github.com/techcodings/Rengy_Full_Stack

Date: April 2026

Author: Rengy

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Database Design
5. RBAC Permission Flow
6. API Endpoints
7. Frontend Pages
8. Project Structure
9. Setup & Installation Guide
10. Seed Data Reference

1. Project Overview

TeamVault is a production-ready, full-stack web application built using the MERN stack (MongoDB, Express.js, React, Node.js) that implements a **team-scoped Role-Based Access Control (RBAC)** system.

The application enables organizations to manage users, teams, roles, and permissions with a dynamic authorization model where a user can hold **different roles in different teams**.

Key Features

- **User Management** — Create users with unique email, search and filter with pagination
- **Team Management** — Create teams, add/remove members, view team compositions
- **Role Management** — Create reusable roles (Admin, Manager, Viewer) with configurable permissions
- **Permission Collection** — Permissions stored as first-class DB entities (not hardcoded strings)
- **Team-Scoped RBAC** — User A can be Admin in Team Alpha but Viewer in Team Beta
- **Permission Resolver** — Dynamic UI to query any user's effective permissions in any team
- **JWT Authentication** — Secure login/register with token-based auth
- **Authorization Middleware —** Reusable `checkPermission('CREATE_TASK')` middleware

2. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	Single-page application with hooks
Styling	Tailwind CSS + Custom CSS	Dark theme with glassmorphism design
Animation	Framer Motion	Page transitions and micro-animations
HTTP Client	Axios	API calls with JWT interceptors
Backend	Node.js + Express.js	REST API server
Database	MongoDB + Mongoose	Document database with ODM
Authentication	JWT + bcryptjs	Token auth with password hashing
Validation	express-validator	Request validation middleware
Security	Helmet + CORS	HTTP headers and cross-origin policy

3. System Architecture

The application follows a **layered architecture** with clear separation of concerns:

Client (React) → Axios + JWT → Express Routes → Middleware → Controllers → Services → MongoDB

Backend Layers

Layer	Responsibility	Example
Routes	Define endpoints, attach middleware	`router.post('/', auth, validate, controller.create)`
Middleware	Auth, authorization, validation, errors	`checkPermission('CREATE_TASK')`
Controllers	Thin — parse request, call service, send response	`const result = await service.create(req.body)`
Services	Business logic, DB queries, validations	`await Role.findById(id).populate('permissions')`
Models	Mongoose schemas, indexes, transforms	`Membership { userId, teamId, roleId }`

4. Database Design

The system uses **5 MongoDB collections** with proper ObjectId references:

Collection	Fields	Purpose
users	name, email (unique), password (hashed)	User accounts
teams	name, description, createdBy	Team entities
permissions	name (unique, UPPER_SNAKE_CASE), description	Permission definitions
roles	name (unique), permissions: [ObjectId→Permission], description	Roles with permission refs
memberships	userId→User, teamId→Team, roleId→Role	User-Team-Role mapping

> **Critical Design Decision:** Roles are NOT stored on the User document. The `Membership` collection maps `(userId, teamId, roleId)`, enabling a user to hold different roles in different teams. This is the industry-standard approach for team-scoped RBAC.

Relationship Diagram

```
User ──────┐
           │  userId
           ▼
      Membership ◄── Compound Unique Index (userId + teamId)
           │
 teamId ───┼─── roleId
           │        │
           ▼        ▼
         Team     Role ──► permissions: [ObjectId]
                                  │
                                  ▼
                          Permission { name, description }
```

5. RBAC Permission Flow

When a protected API request is made, the authorization follows this flow:

1. JWT Token → 2. auth.js → 3. checkPermission() → 4. Membership Lookup →

5. Role → Permissions → 6. Allow/Deny

1. Client sends request with `Authorization: Bearer <JWT> + teamId`
2. `auth.js` middleware verifies JWT → attaches `req.user`
3. `checkPermission("CREATE_TASK")` middleware runs
4. Looks up `Membership` for `(userId, teamId)`
5. Deep populates: `Role → Permission[]` → extracts permission names
6. If permission found → `next()` | If not → `403 Forbidden`

Usage Example

```
// In route definition:
router.post('/tasks',
  auth,                            // Step 1-2: Verify JWT
  checkPermission('CREATE_TASK'),  // Step 3-6: Check permission
  taskController.createTask        // Handler runs only if authorized
);
```

6. API Endpoints

Authentication

Method	Endpoint	Description
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login → returns JWT
GET	/api/auth/me	Get current user profile

Users

Method	Endpoint	Description
POST	/api/users	Create user
GET	/api/users?search=&page=&limit=	List users (search + pagination)

Teams & Membership

Method	Endpoint	Description

POST	/api/teams	Create team
GET	/api/teams	List teams
POST	/api/teams/:id/add-user	Add member to team
DELETE	/api/teams/:id/remove-user	Remove member from team
GET	/api/teams/:id/members	List team members

Roles & Permissions

Method	Endpoint	Description
POST	/api/roles	Create role (with Permission ObjectIds)
GET	/api/roles	List roles (populated permissions)
POST	/api/permissions	Create permission
GET	/api/permissions/constants	List all permissions
GET	/api/permissions?userId=&teamId=	Resolve user permissions in team
POST	/api/assign-role	Assign role to user in team
PUT	/api/update-role	Update role assignment

7. Frontend Pages

Page	Route	Features
Login	/login	Email/password, JWT auth, demo credentials
Register	/register	Name, email, password with validation
Dashboard	/	Stat cards (users, teams, roles), recent lists
Users	/users	User list, search by name/email, pagination, create modal
Teams	/teams	Team list, expandable members panel, add/remove, role badges
Roles	/roles	Role cards with color-coded permission badges, create modal
Permissions	/permissions	User+Team dropdowns → dynamic GRANTED/DENIED cards

8. Project Structure

```
Rengy_Full_Stack/
├── .gitignore
├── README.md
│
├── server/                        # Backend (Express + MongoDB)
│   ├── .env.example               # Environment template
│   ├── package.json
│   ├── server.js                  # Entry point
│   ├── seed.js                    # Database seeder
│   └── src/
│       ├── app.js                 # Express app setup
│       ├── config/
│       │   ├── db.js              # MongoDB connection
│       │   └── env.js             # Environment variables
│       ├── constants/
│       │   └── permissions.js     # Default permission seeds
│       ├── models/
│       │   ├── User.js
│       │   ├── Team.js
│       │   ├── Role.js
│       │   ├── Permission.js
│       │   └── Membership.js
│       ├── services/              # Business logic layer
│       ├── controllers/           # Request handlers (thin)
│       ├── routes/                # API route definitions
```

```
│       ├── middleware/
│       │   ├── auth.js              # JWT verification
│       │   ├── authorize.js         # checkPermission()
│       │   ├── errorHandler.js      # Global error handler
│       │   └── validate.js          # Validation runner
│       ├── validators/              # express-validator chains
│       └── utils/
│           ├── ApiError.js          # Custom error class
│           ├── ApiResponse.js       # Standardized responses
│           └── jwt.js               # Token helpers
│
└── client/                          # Frontend (React + Vite)
    ├── package.json
    ├── vite.config.js               # Vite + proxy config
    └── src/
        ├── App.jsx                  # Root with routing
        ├── main.jsx                 # Entry point
        ├── index.css                # Global dark theme styles
        ├── api/axios.js             # Axios with JWT interceptors
        ├── context/AuthContext.jsx   # Authentication state
        ├── hooks/useApi.js          # Reusable fetch hook
        ├── components/
        │   ├── layout/              # Sidebar, Navbar, AppLayout
        │   └── ui/                  # Modal, Loader
        └── pages/                   # 7 page components
```

9. Setup & Installation Guide

Prerequisites

- **Node.js** v18 or higher — nodejs.org
- **MongoDB** running locally on port 27017 — mongodb.com
- **Git** installed — git-scm.com

Step 1: Clone the Repository

```
git clone https://github.com/techcodings/Rengy_Full_Stack.git
cd Rengy_Full_Stack
```

Step 2: Setup Backend

```
cd server
npm install
```

Step 3: Configure Environment Variables

```
# Copy the example env file
cp .env.example .env

# The .env file should contain:
PORT=5000
MONGODB_URI=mongodb://localhost:27017/teamvault
JWT_SECRET=your_jwt_secret_key_here
JWT_EXPIRES_IN=7d
NODE_ENV=development
```

Step 4: Start MongoDB

```
# Make sure MongoDB is running on localhost:27017
# On Windows: net start MongoDB
# On Mac: brew services start mongodb-community
# On Linux: sudo systemctl start mongod
```

Step 5: Seed the Database

```
npm run seed
```

This creates: **4 permissions**, **3 roles**, **5 users**, **3 teams**, and **9 memberships**.

Step 6: Start the Backend Server

```
npm run dev
```

Server runs at `http://localhost:5000`

Step 7: Setup Frontend (New Terminal)

```
cd ../client
npm install
npm run dev
```

Frontend runs at `http://localhost:3000`

Step 8: Open the Application

```
# Open in browser:
http://localhost:3000

# Login with seed credentials:
Email:    alice@example.com
Password: password123
```

> **Quick Start (All Commands):**
> `git clone https://github.com/techcodings/Rengy_Full_Stack.git && cd Rengy_Full_Stack`
> `cd server && npm install && cp .env.example .env && npm run seed && npm run dev`
> *(Open new terminal)*
> `cd client && npm install && npm run dev`

10. Seed Data Reference

Permissions

Permission	Description
CREATE_TASK	Create new tasks and assignments
EDIT_TASK	Modify existing tasks and details

DELETE_TASK	Permanently remove tasks
VIEW_ONLY	Read-only access to team content

Roles

Role	Permissions
Admin	CREATE_TASK, EDIT_TASK, DELETE_TASK, VIEW_ONLY
Manager	CREATE_TASK, EDIT_TASK, VIEW_ONLY
Viewer	VIEW_ONLY

User-Team-Role Mapping

User	Team Alpha	Team Beta	Team Gamma
Alice Johnson	Admin	Viewer	—
Bob Smith	Manager	Admin	—
Charlie Brown	Viewer	—	Admin
Diana Prince	—	Manager	Viewer
Eve Williams	—	—	Manager

> **All login credentials:** Use any email above with password `password123`

TeamVault — Team Management System with RBAC · Built with MERN Stack · github.com/techcodings/Rengy_Full_Stack